

Multi Image Picker with ListView Documentation

Overview

This document explains how to **pick multiple images** from the device gallery and **display them in a ListView** in Flutter.

The goal is to: - Select **multiple images at once** - Store them in a list - Show the selected images dynamically in a `ListView`

Use Case

- Profile image gallery
 - Product image upload
 - Post creation with multiple photos
 - Chat media preview
-

Required Package

We use the popular Flutter package:

```
dependencies:  
  image_picker: ^1.0.7
```

Step 1: State Variables

These variables store selected images.

```
import 'dart:io';  
import 'package:image_picker/image_picker.dart';  
  
final ImagePicker _picker = ImagePicker();  
List<XFile> selectedImages = [];
```

Step 2: Pick Multiple Images Function

This function opens the gallery and lets the user select **multiple images**.

```
Future<void> pickImages() async {  
  final List<XFile>? images = await _picker.pickMultiImage();  
  
  if (images != null && images.isNotEmpty) {  
    setState(() {  
      selectedImages.addAll(images);  
    });  
  }  
}
```

What happens here

- Opens gallery
- User selects multiple images
- Images are added to `selectedImages` list
- UI updates automatically

Step 3: Display Images in ListView

We use `ListView.builder` to show selected images.

```
ListView.builder(  
  itemCount: selectedImages.length,  
  itemBuilder: (context, index) {  
    return Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Image.file(  
        File(selectedImages[index].path),  
        height: 120,  
        fit: BoxFit.cover,  
      ),  
    );  
  },  
);
```

Step 4: Full UI Example

```
class MultiImagePickerScreen extends StatefulWidget {
  const MultiImagePickerScreen({super.key});

  @override
  State<MultiImagePickerScreen> createState() => _MultiImagePickerScreenState();
}

class _MultiImagePickerScreenState extends State<MultiImagePickerScreen> {
  final ImagePicker _picker = ImagePicker();
  List<XFile> selectedImages = [];

  Future<void> pickImages() async {
    final List<XFile>? images = await _picker.pickMultiImage();

    if (images != null && images.isNotEmpty) {
      setState(() {
        selectedImages.addAll(images);
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Multi Image Picker')),
      floatingActionButton: FloatingActionButton(
        onPressed: pickImages,
        child: const Icon(Icons.add),
      ),
      body: selectedImages.isEmpty
        ? const Center(child: Text('No images selected'))
        : ListView.builder(
            itemCount: selectedImages.length,
            itemBuilder: (context, index) {
              return Padding(
                padding: const EdgeInsets.all(8.0),
                child: Image.file(
                  File(selectedImages[index].path),
                  height: 120,
                  fit: BoxFit.cover,
                ),
              );
            },
          ),
    );
  }
}
```

```
    );  
  }  
}
```

Key Points

- `pickMultiImage()` allows selecting multiple images
 - Images are stored as `XFile`
 - `ListView.builder` efficiently renders images
 - UI updates using `setState`
-

Optional Improvements

- Add remove image button
 - Limit maximum image count
 - Use `GridView` instead of `ListView`
 - Compress images before upload
-

Summary

This approach provides a clean and scalable way to select multiple images and display them dynamically in a `ListView`. It works with static UI and can easily be connected to APIs for uploads.

You can freely edit this documentation as needed.